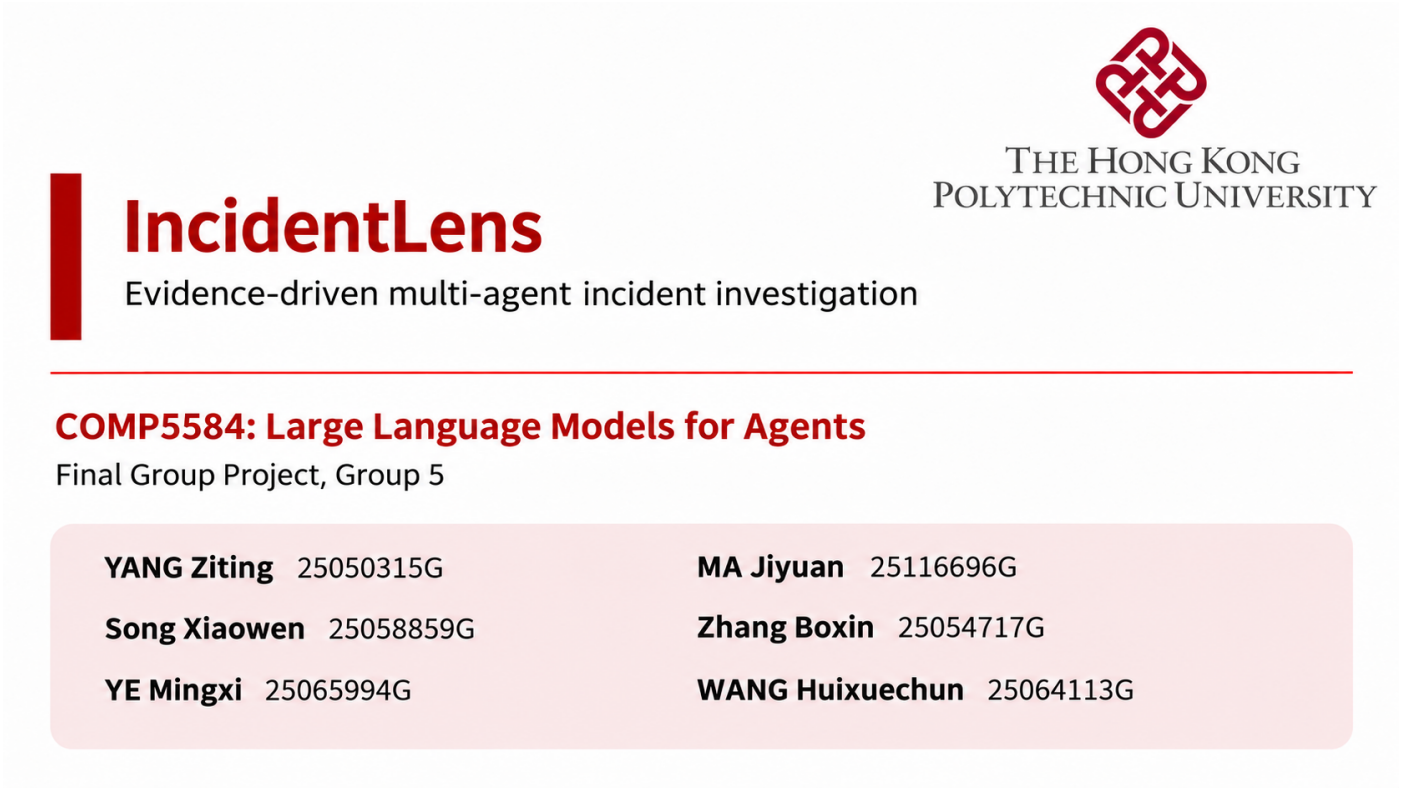
Slide 1: IncidentLens

Main presentation - Speaker block: S1

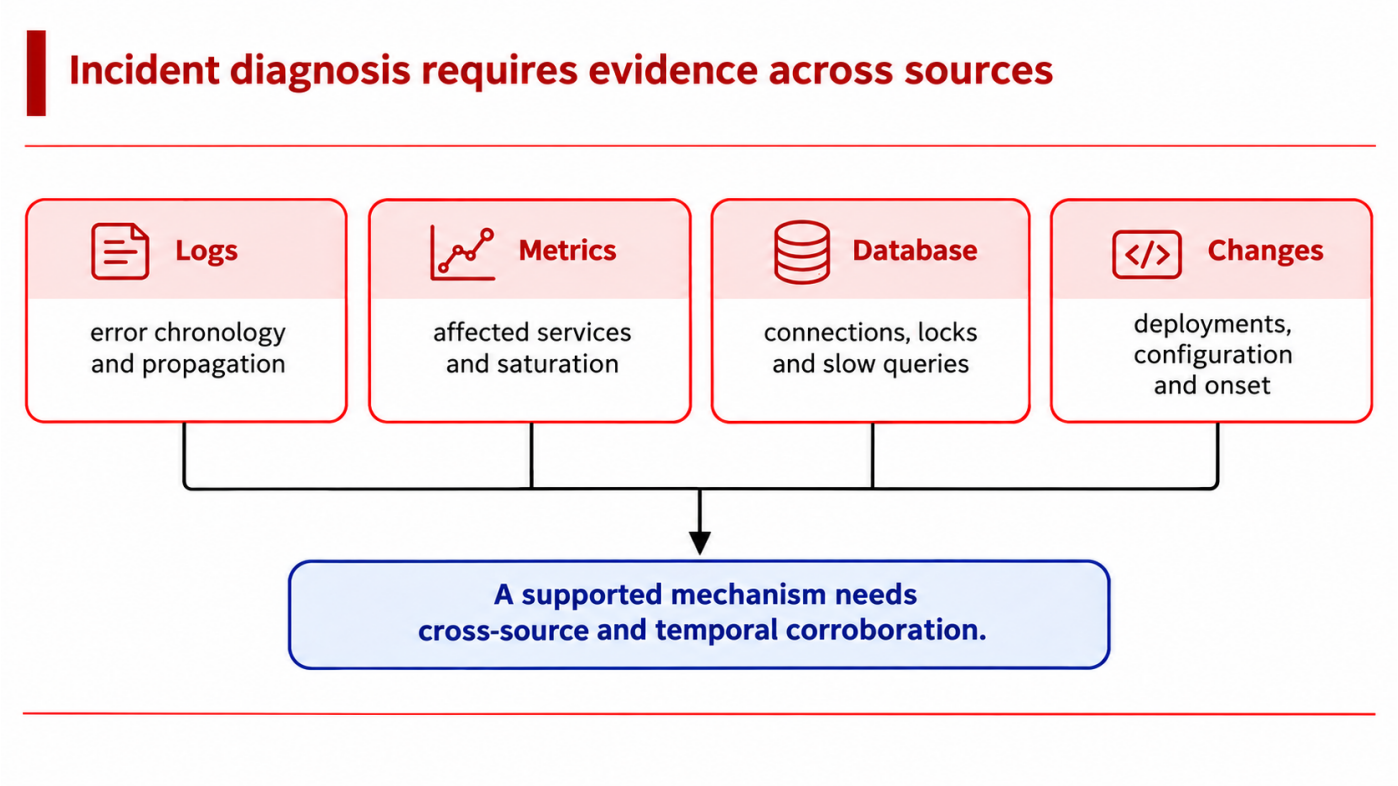


Presenter script

Good morning. We are Group Five, and our project is IncidentLens, a multi-agent system for investigating software incidents. An accepted root cause must connect to evidence the system actually observed. When the evidence is insufficient, the system can explicitly abstain. We will explain the collaboration design, demonstrate the system, and report a matched comparison of four architectures.

Slide 2: Incident diagnosis requires evidence across sources

Main presentation - Speaker block: S1



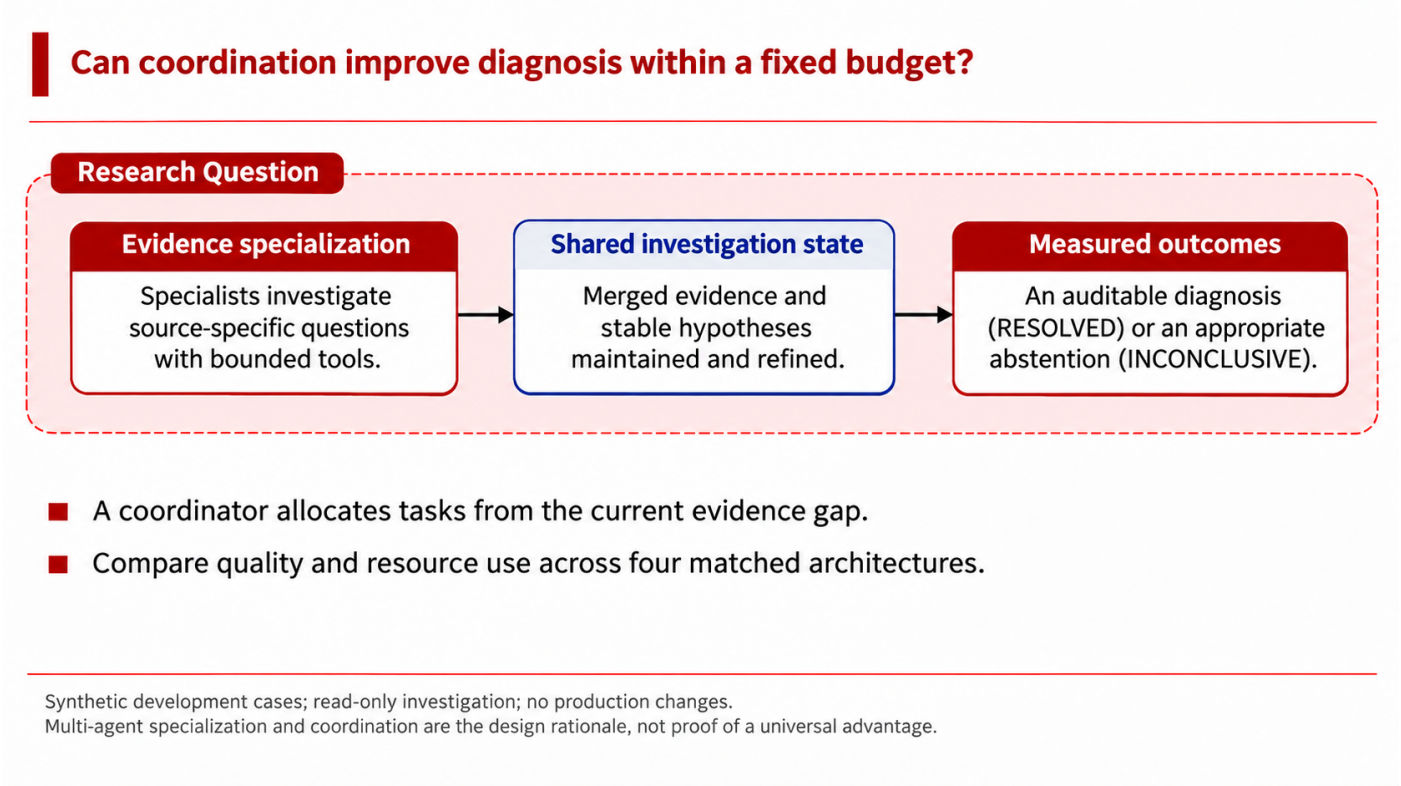
Presenter script

An incident usually leaves several partial views. Logs show which errors appeared and in what order. Metrics show the affected services and signs of saturation. Database observations reveal sessions, locks or slow queries. Change records help us identify what happened around the start of the failure.

The difficult step is connecting these observations into a supported mechanism. A deployment near the incident may be relevant, but timing alone does not establish the cause. Our system therefore investigates across sources and keeps the remaining uncertainty visible.

Slide 3: Can coordination improve diagnosis within a fixed budget?

Main presentation - Speaker block: S1



Presenter script

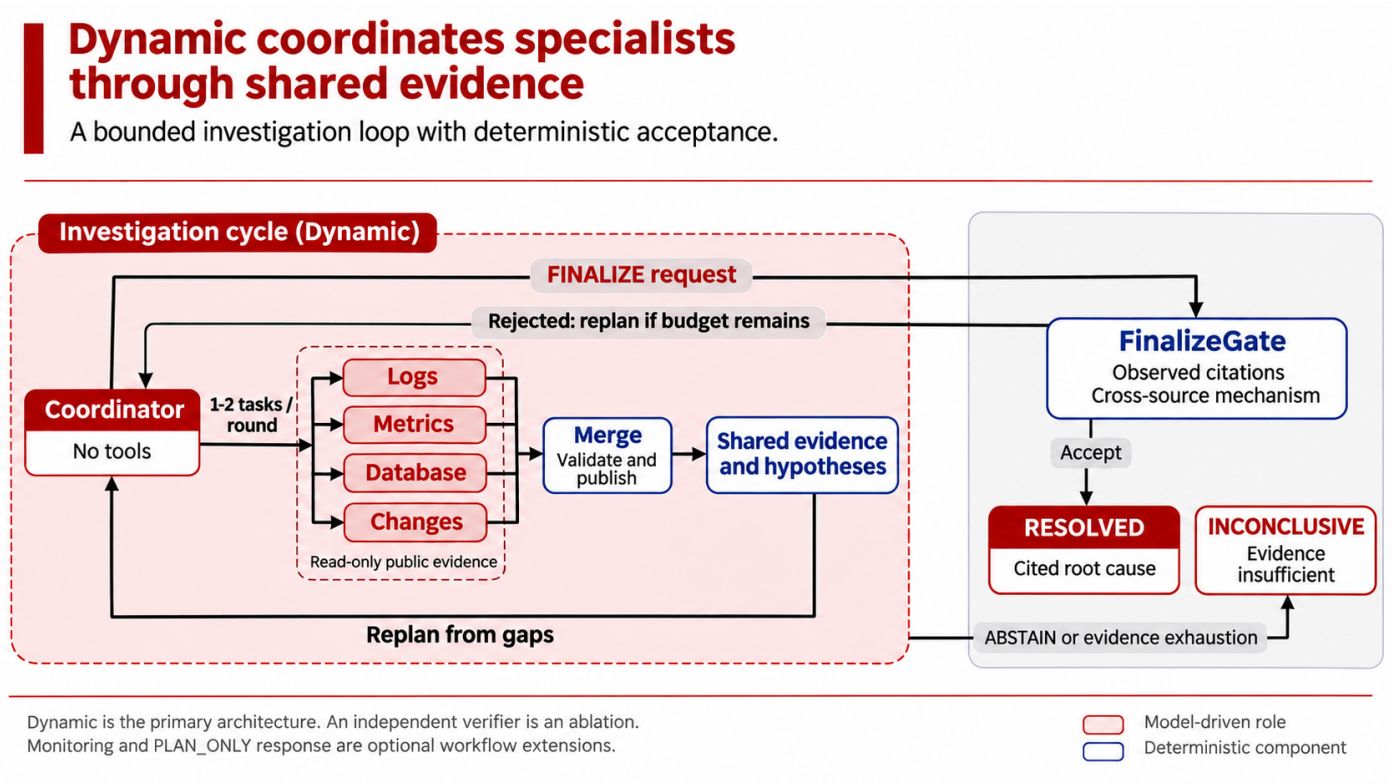
Our research question is whether coordination can produce an auditable diagnosis, or an appropriate abstention, within a fixed investigation budget.

The roles have different questions and different tools. A coordinator decides which evidence gap to address next, while specialists investigate only their assigned scope. Shared evidence lets their work accumulate across rounds.

We contribute this bounded coordination design, deterministic acceptance checks, and a matched evaluation against three controls. The scope is a research prototype over synthetic incidents. I will now hand over to explain how the agents collaborate.

Slide 4: Dynamic coordinates specialists through shared evidence

Main presentation - Speaker block: S2



Presenter script

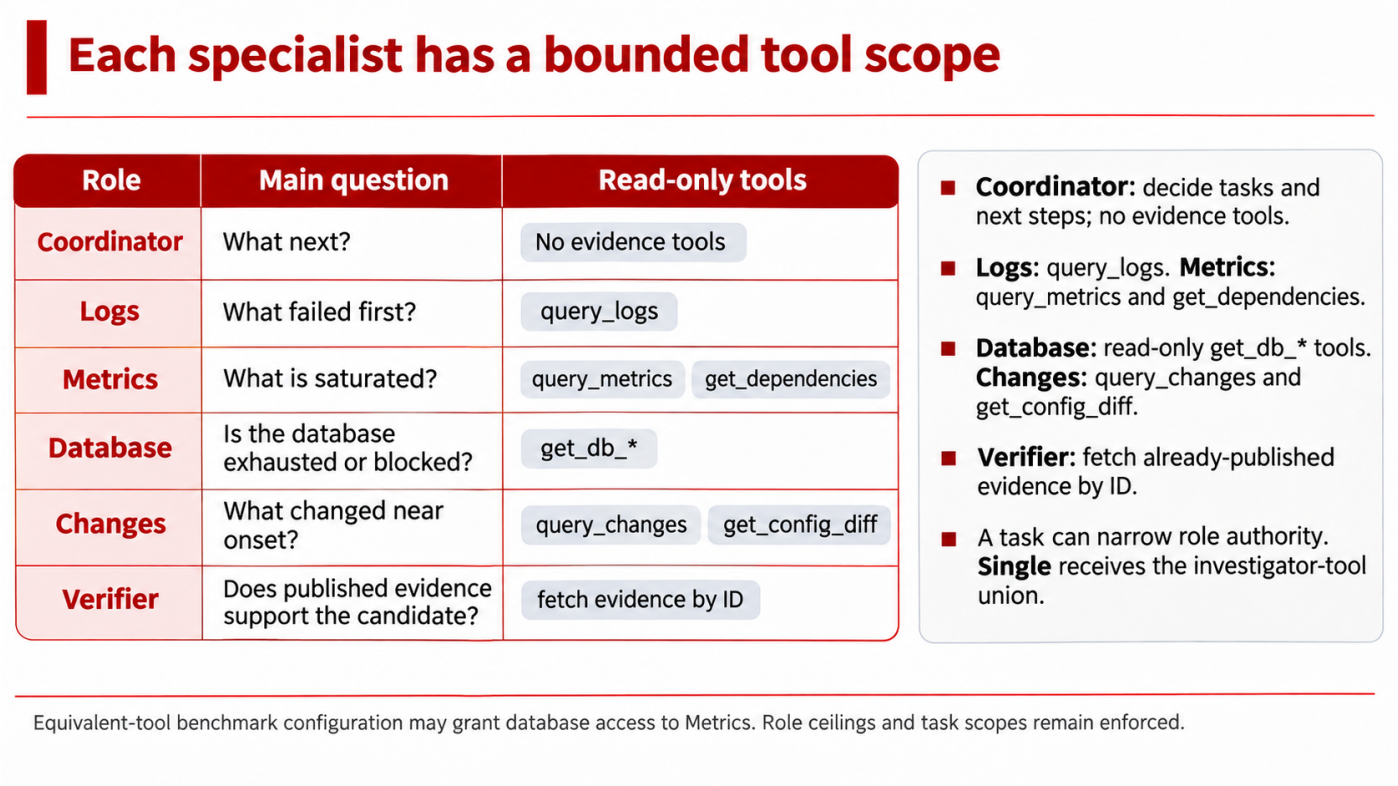
The coordinator chooses one or two specialist tasks from the current evidence gap. Each specialist has read-only tools for its source and returns a local finding. Merge validates identity and citations, then updates the shared evidence and hypothesis board.

The coordinator uses that board to investigate further or request finalization. FinalizeGate checks the candidate in code. Acceptance produces a cited root cause. Rejection can return the run to investigation while budget remains. The coordinator can also stop with an inconclusive result when evidence is insufficient.

Dynamic has no independent verifier. We evaluate that extra reviewer separately as an ablation.

Slide 5: Each specialist has a bounded tool scope

Main presentation - Speaker block: S2



Presenter script

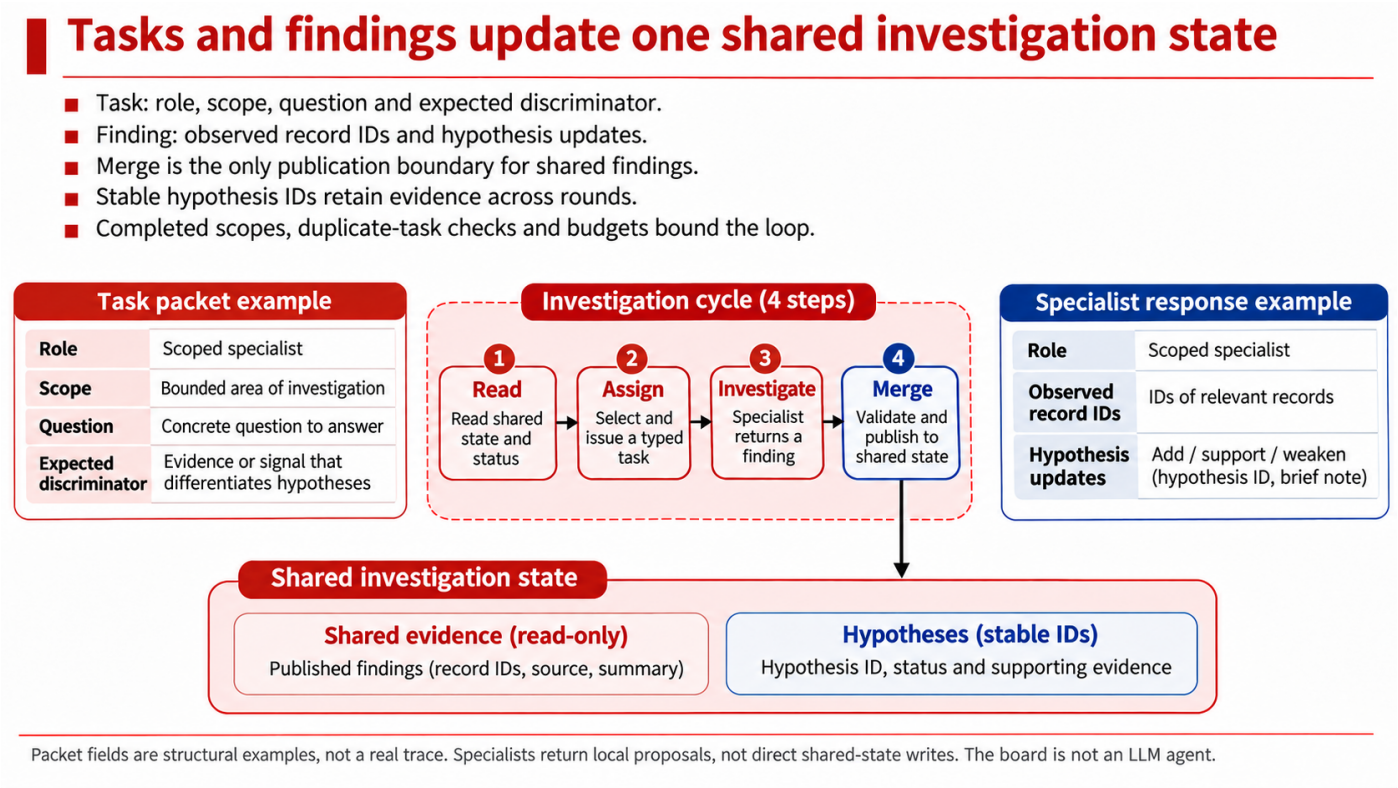
The role boundaries are enforced by the runtime. The coordinator has no evidence tools. The logs role queries logs, while the other specialists receive tools for metrics, database observations or changes.

Each task can narrow its role's permissions. It cannot increase them. The verifier can fetch evidence already published to the shared board, but it cannot start a fresh investigation.

For the comparison, the Single baseline receives the union of investigator tools. This gives it broad access, so the comparison tests more than a difference in available evidence.

Slide 6: Tasks and findings update one shared investigation state

Main presentation - Speaker block: S2



Presenter script

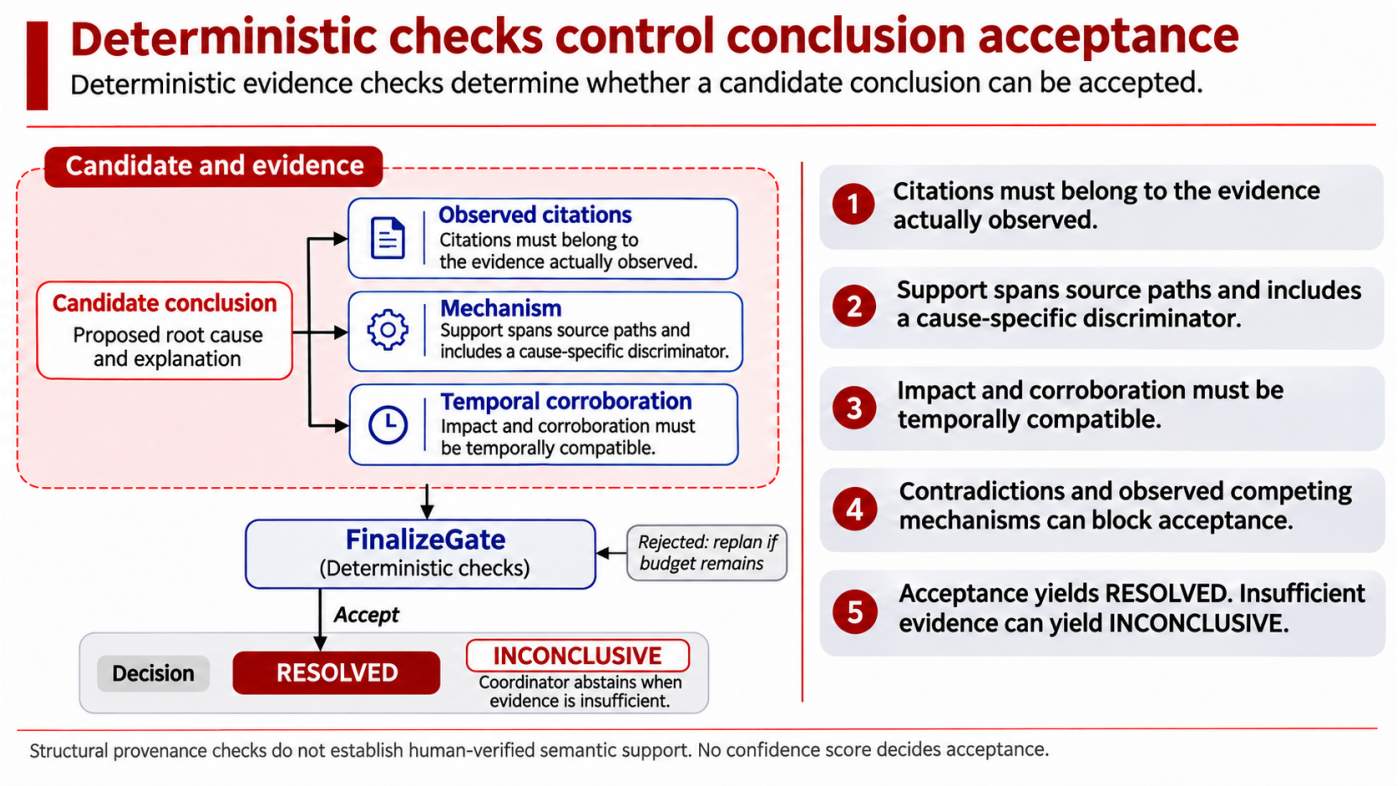
Communication uses structured tasks and findings. A task specifies the role, scope, investigative question and expected discriminator. A specialist returns findings with record identifiers and proposed hypothesis updates.

Merge checks these local results before publishing them. A stable hypothesis identifier keeps evidence attached to the same candidate across rounds, even when the explanation changes.

The coordinator then sees the updated hypotheses, remaining gaps and budget. Completed scopes and repeated task signatures help prevent unproductive loops. This is how the system turns several specialist calls into one continuing investigation.

Slide 7: Deterministic checks control conclusion acceptance

Main presentation - Speaker block: S3



Presenter script

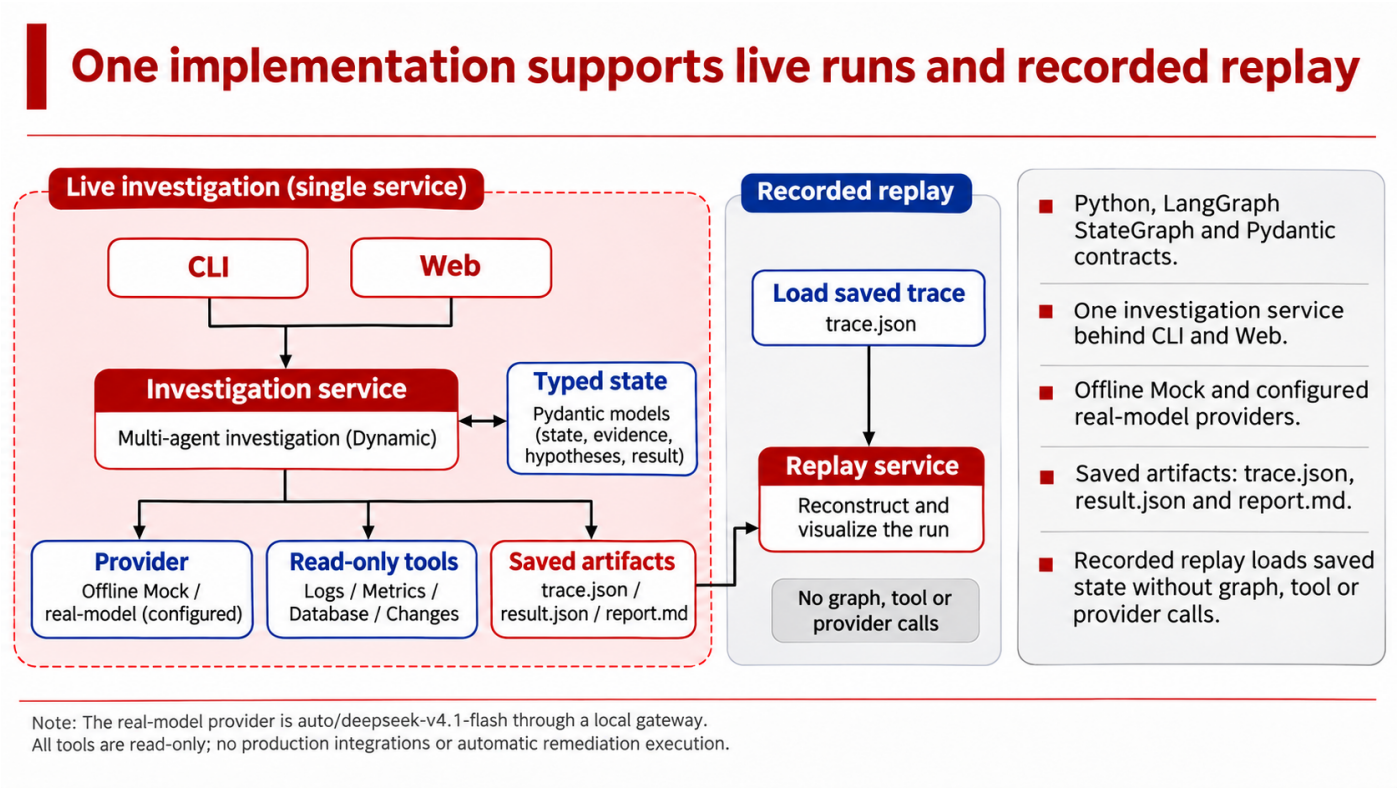
A candidate needs more than a confident explanation. Its cited records must exist in the observed evidence. Supporting records must span different source paths, include a cause-specific discriminator, and provide temporally compatible corroboration.

Contradictory evidence and an observed competing mechanism can block acceptance. The model cannot override those checks by increasing its confidence.

These rules establish structural provenance and acceptance conditions. They do not prove that every sentence has perfect semantic support. When the available evidence cannot satisfy the conditions, the report keeps the result inconclusive. Next, let us look at the implementation and interface.

Slide 8: One implementation supports live runs and recorded replay

Main presentation - Speaker block: S3



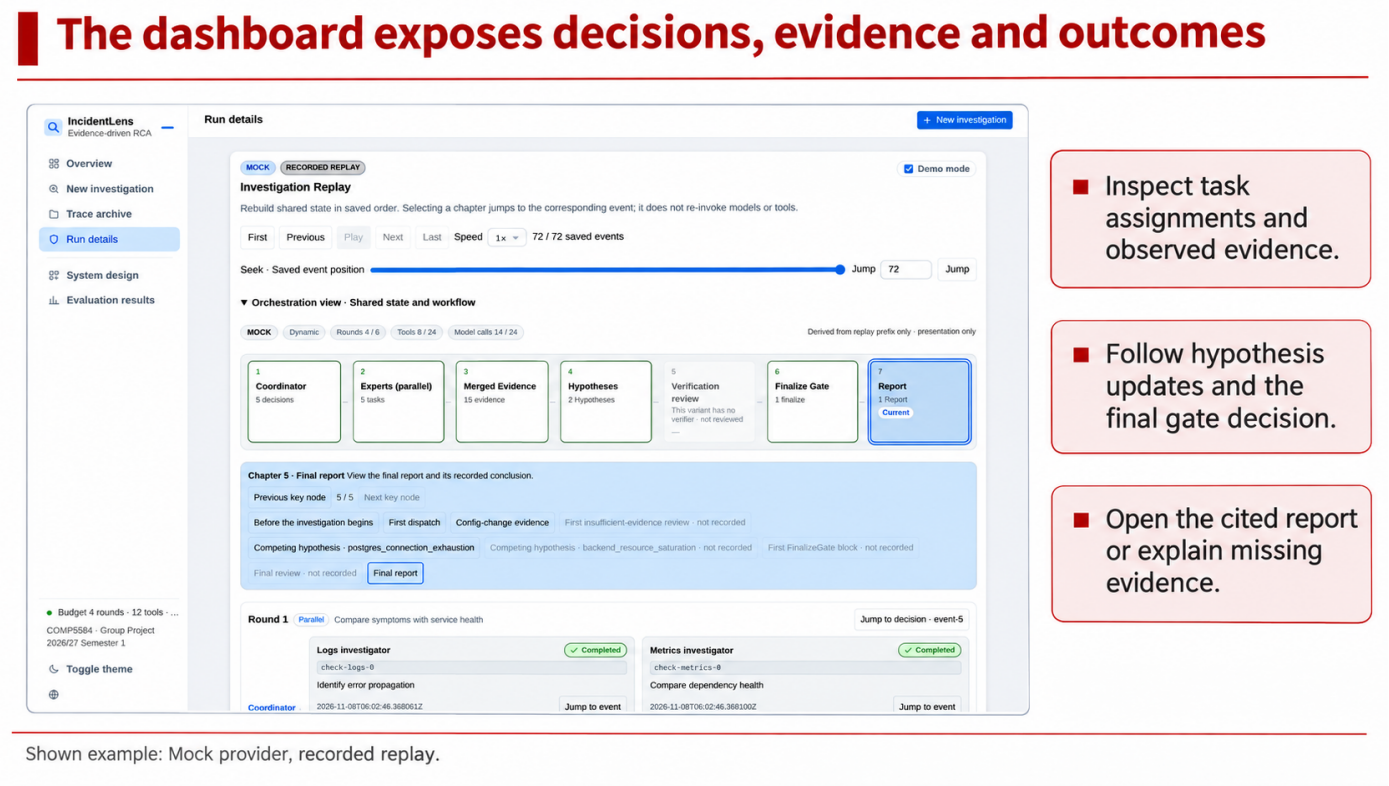
Presenter script

The implementation uses Python, LangGraph for orchestration, and Pydantic for typed contracts. The same investigation service supports the command line and the web dashboard. It can use an offline Mock provider or a configured real-model provider.

Each run saves its trace, result and report. Recorded replay reconstructs the saved state without invoking the graph, tools or model again. This gives us an inspectable artifact for evaluation and a reliable demonstration fallback. We will use that interface to show how the investigation becomes visible.

Slide 9: The dashboard exposes decisions, evidence and outcomes

Main presentation - Speaker block: S3



Shown example: Mock provider, recorded replay.

Presenter script

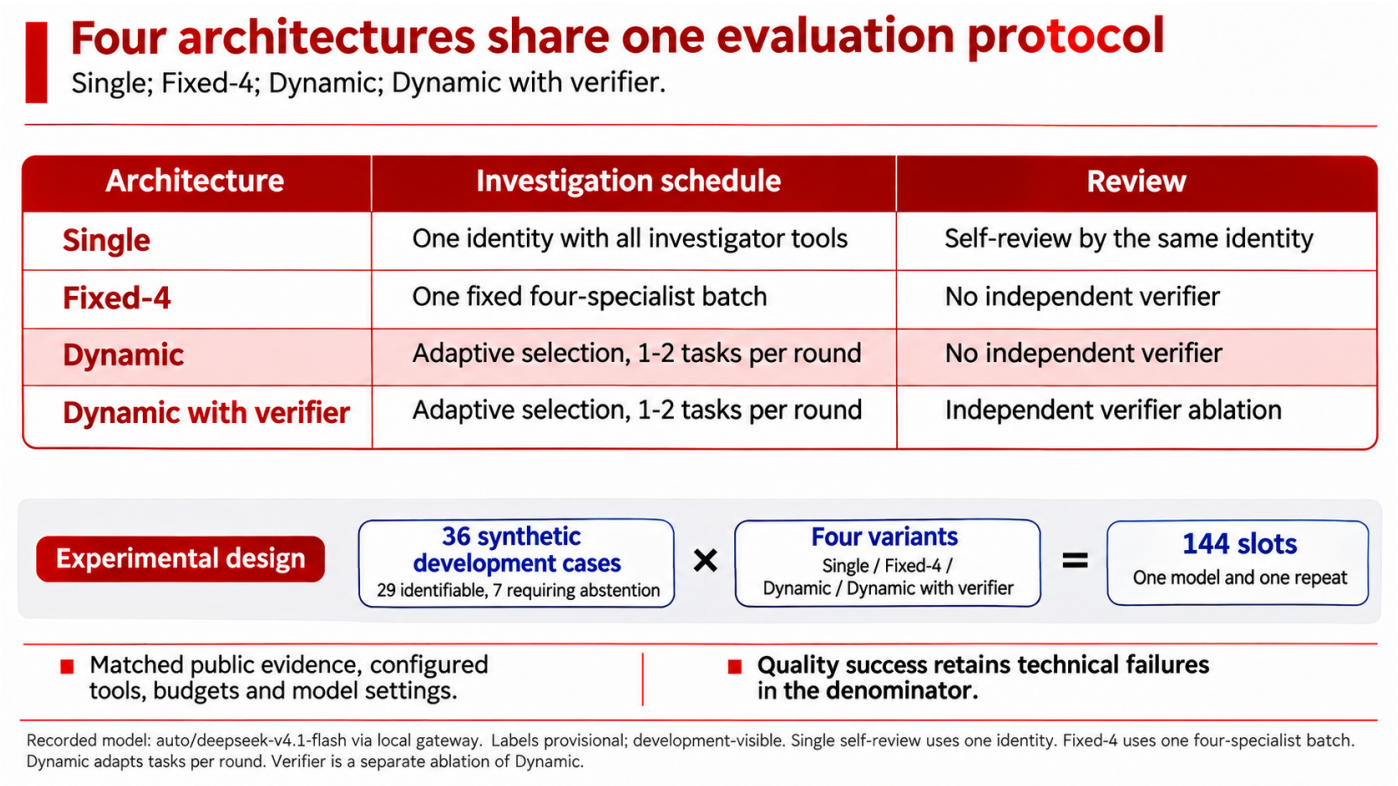
This dashboard makes the investigation inspectable. The example shown is a Mock recorded replay. Its labels distinguish saved playback from a live run.

During the demonstration, follow the coordinator's task assignments, the specialists' observed evidence, and the hypothesis board. Then inspect the final gate decision and its cited report. We can also load an inconclusive example to see which evidence remains missing.

The same saved trace supports this inspection without another model call. With the workflow established, we can now compare the architectures systematically.

Slide 10: Four architectures share one evaluation protocol

Main presentation - Speaker block: S4



Presenter script

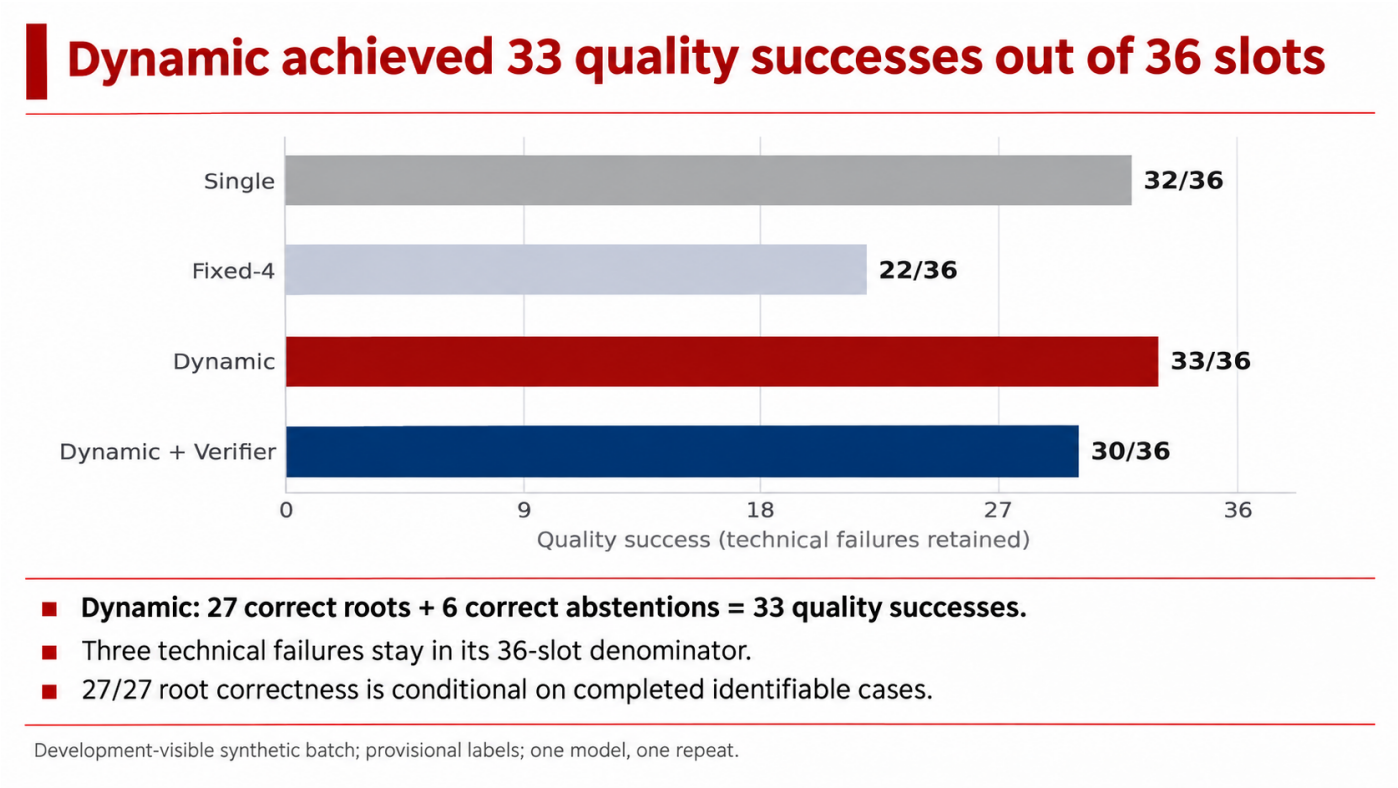
We compare Single, Fixed-Four, Dynamic, and Dynamic with an independent verifier. The protocol matches the model, public evidence, configured tool access and budgets.

The hardened development set contains thirty-six synthetic cases: twenty-nine with identifiable causes and seven requiring abstention. One repeat across four architectures produces one hundred and forty-four slots. The measured model is DeepSeek version four point one Flash through a local gateway.

Our main quality measure includes technical failures in its denominator. Root correctness and correct abstention use their own eligible completed cases. This distinction matters when interpreting the results.

Slide 11: Dynamic achieved 33 quality successes out of 36 slots

Main presentation - Speaker block: S4



Presenter script

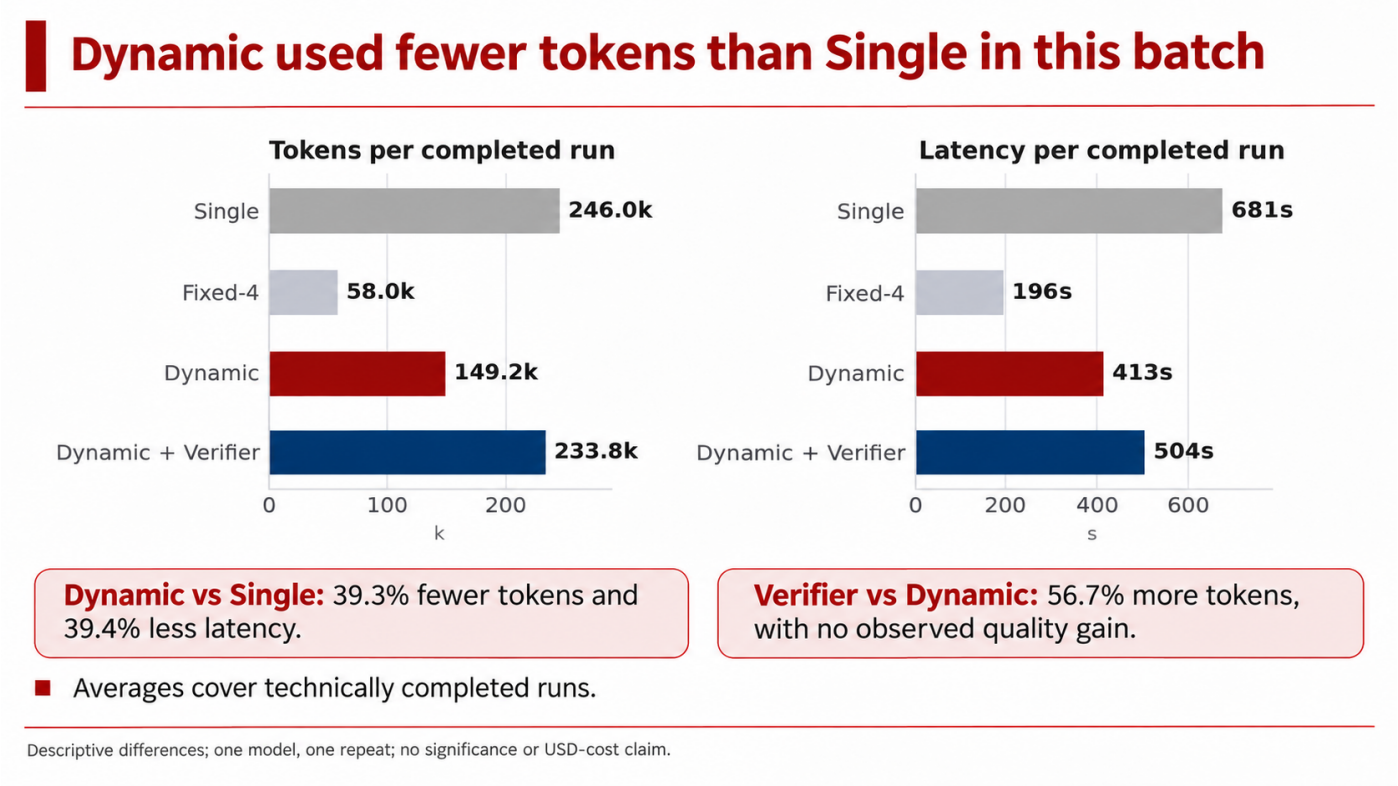
Dynamic achieved thirty-three quality successes out of thirty-six slots, compared with thirty-two for Single, twenty-two for Fixed-Four, and thirty for Dynamic with a verifier.

For Dynamic, the thirty-three successes consist of twenty-seven correct roots and six correct abstentions. Its other three slots failed technically. Therefore, twenty-seven out of twenty-seven describes root correctness among completed identifiable cases. It is not the overall success rate.

Dynamic had the highest measured quality in this batch. The next question is how much computation each architecture used to reach its results.

Slide 12: Dynamic used fewer tokens than Single in this batch

Main presentation - Speaker block: S4



Presenter script

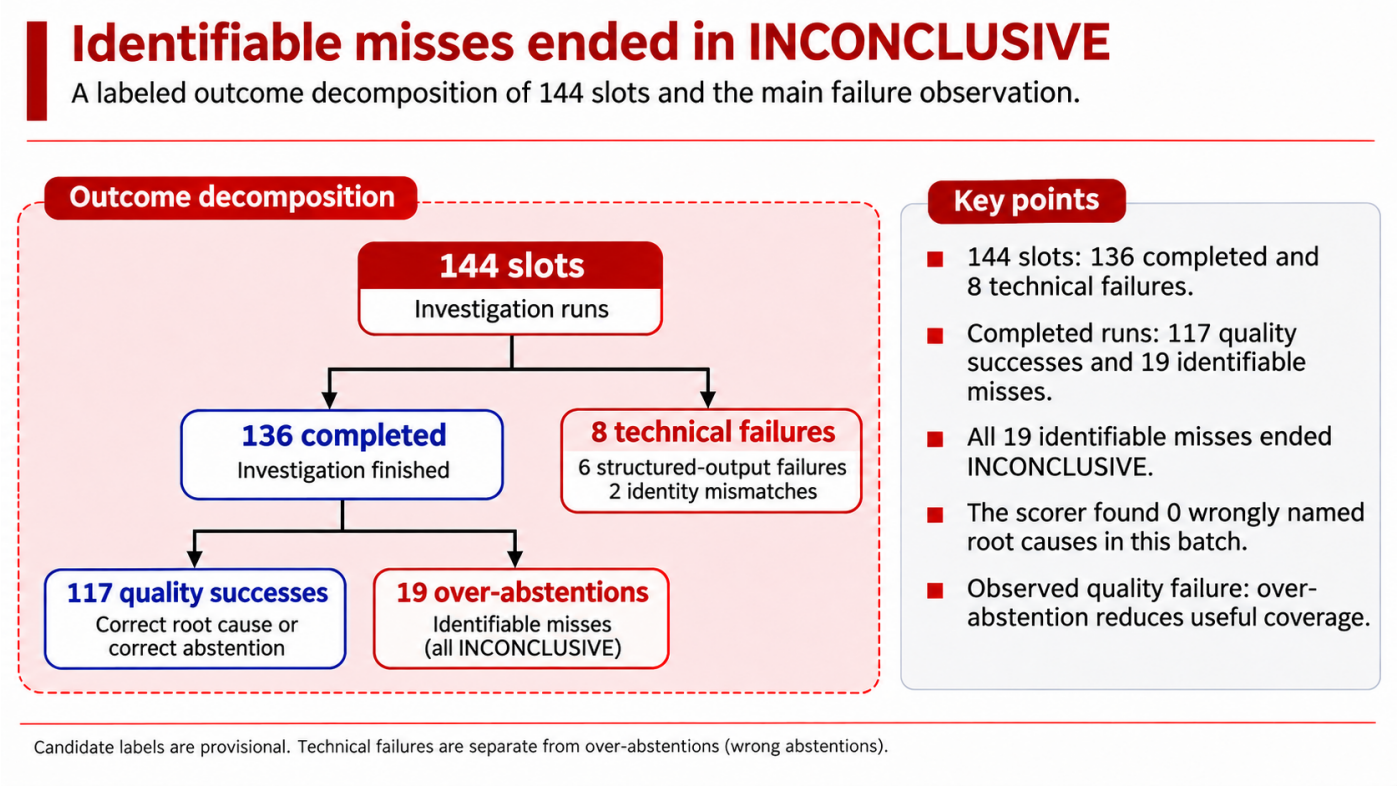
Among completed runs, Dynamic averaged about one hundred and forty-nine thousand tokens and four hundred and thirteen seconds. Compared with Single, that is thirty-nine point three percent fewer tokens and thirty-nine point four percent less latency.

The verifier variant used fifty-six point seven percent more tokens than Dynamic and achieved fewer quality successes. We observed no quality gain from that extra reviewer in this batch.

These are descriptive averages from one model and one repeat. They do not establish statistical significance or dollar cost. We also need to understand the failures behind the aggregate scores.

Slide 13: Identifiable misses ended in INCONCLUSIVE

Main presentation - Speaker block: S5



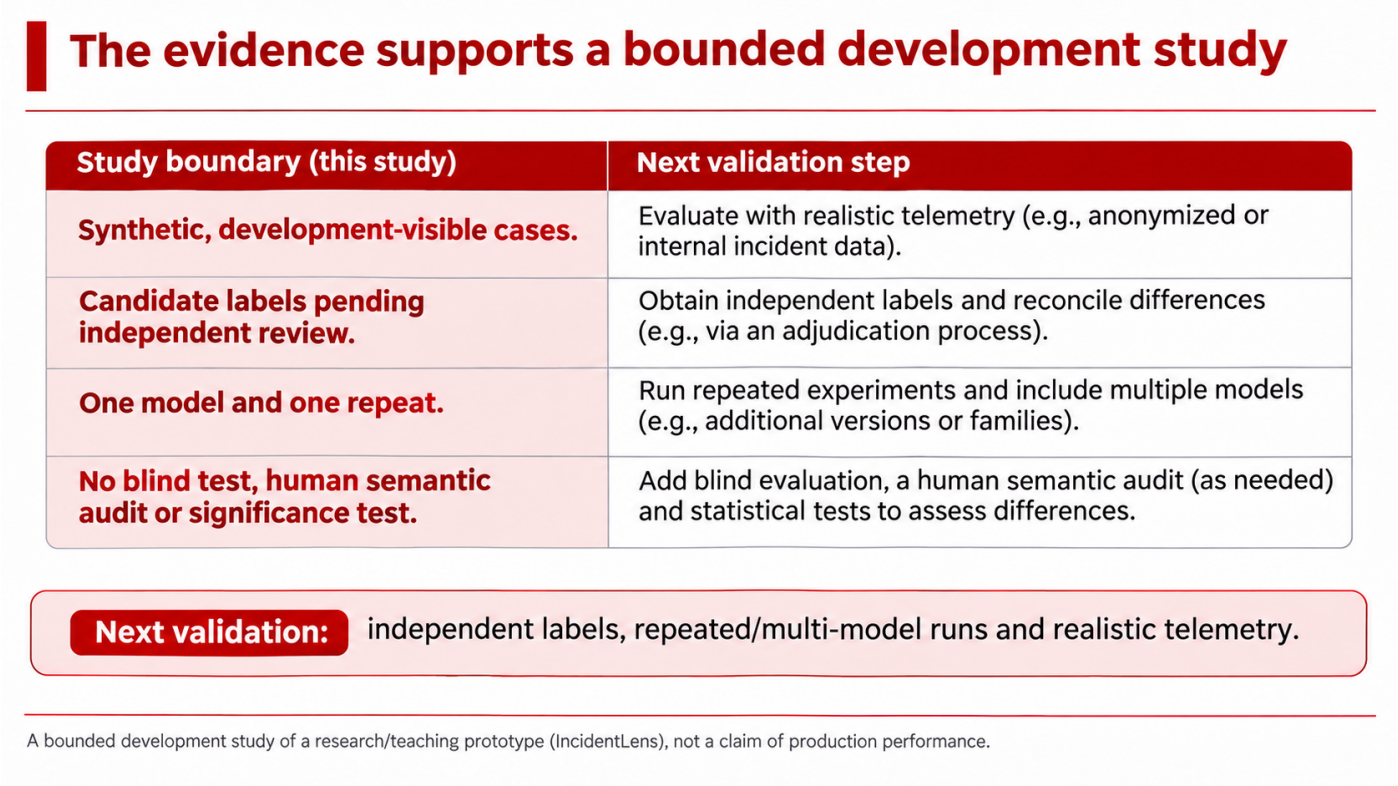
Presenter script

Across all four architectures, one hundred and thirty-six slots completed and eight failed technically. Among completed runs, nineteen identifiable cases ended without a confirmed root cause. All nineteen were inconclusive. The scorer found no wrongly named root causes in this batch.

This means the observed quality failure was over-abstention: the system sometimes refused a case that the candidate labels considered identifiable. That is still a loss of useful coverage. It motivates improving evidence collection and discrimination while preserving the existing acceptance conditions.

Slide 14: The evidence supports a bounded development study

Main presentation - Speaker block: S5



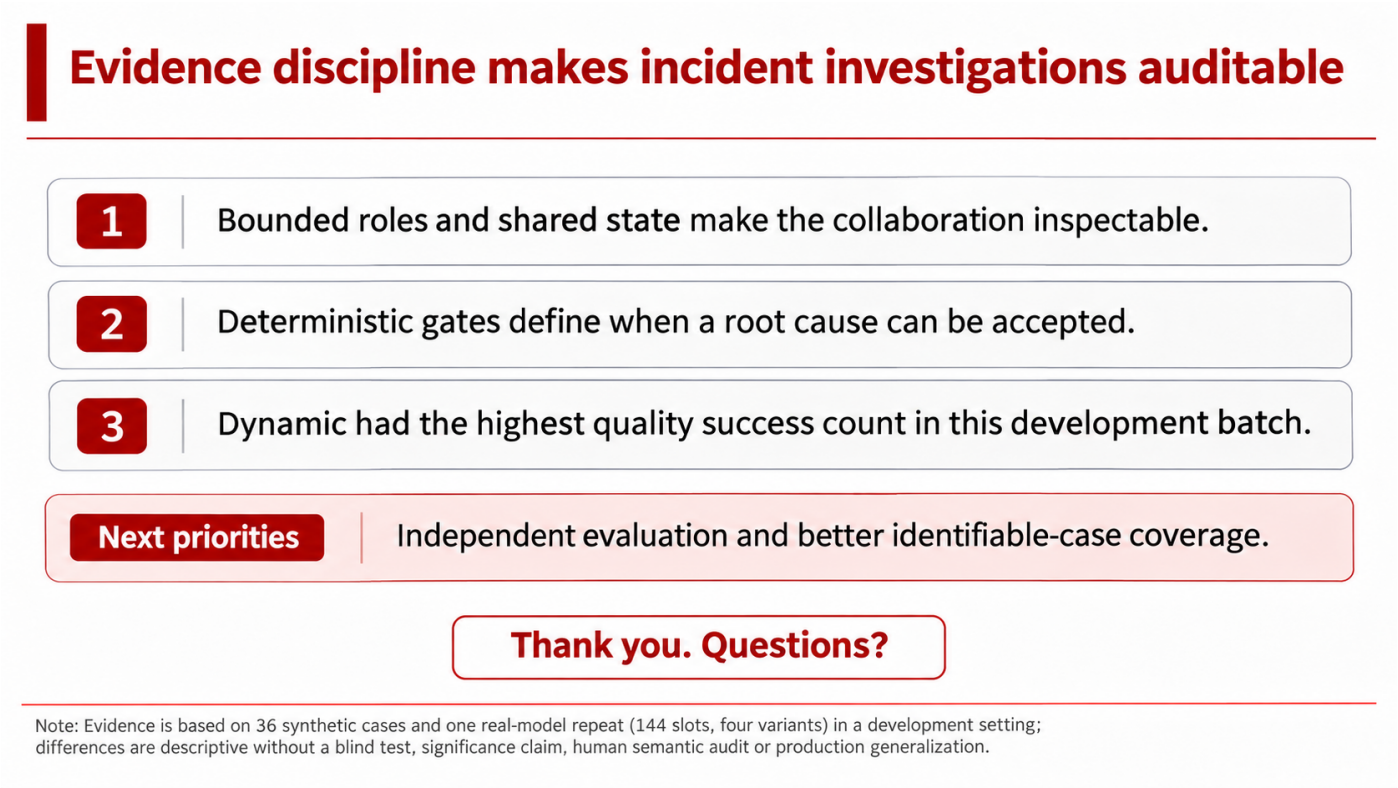
Presenter script

Several boundaries limit what we can conclude. The cases are synthetic and development-visible. Their candidate labels are still pending independent review. We used one model and one repeat, without a blind test or a human audit of semantic support.

The results describe this recorded batch. Independent labeling, repeated runs across models, and more realistic telemetry are the next validation steps. We also need to measure whether changes reduce over-abstention without introducing incorrect conclusions. I will hand over for our final takeaway.

Slide 15: Evidence discipline makes incident investigations auditable

Main presentation - Speaker block: S6



Presenter script

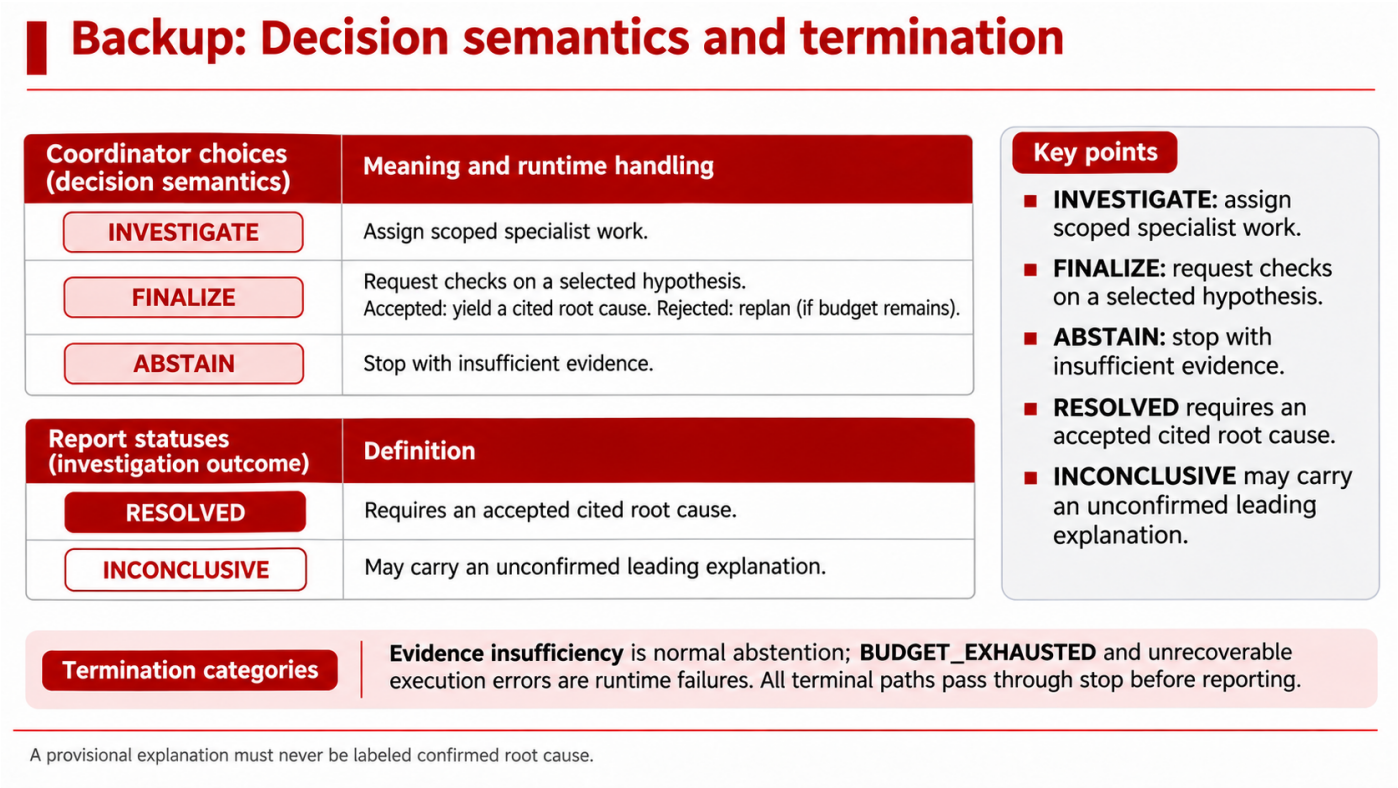
Our main takeaway is that the collaboration can be inspected. Roles have bounded tools, findings enter a shared evidence board through validation, and deterministic checks control conclusion acceptance.

In our development batch, Dynamic achieved the highest quality success count and used fewer tokens and less time than Single. The independent verifier added cost without an observed quality gain.

Our next priorities are stronger independent evaluation and better coverage on identifiable cases. Thank you. We are ready for questions.

Slide 16: Decision semantics and termination

Backup: use during questions - Speaker block: Q&A backup



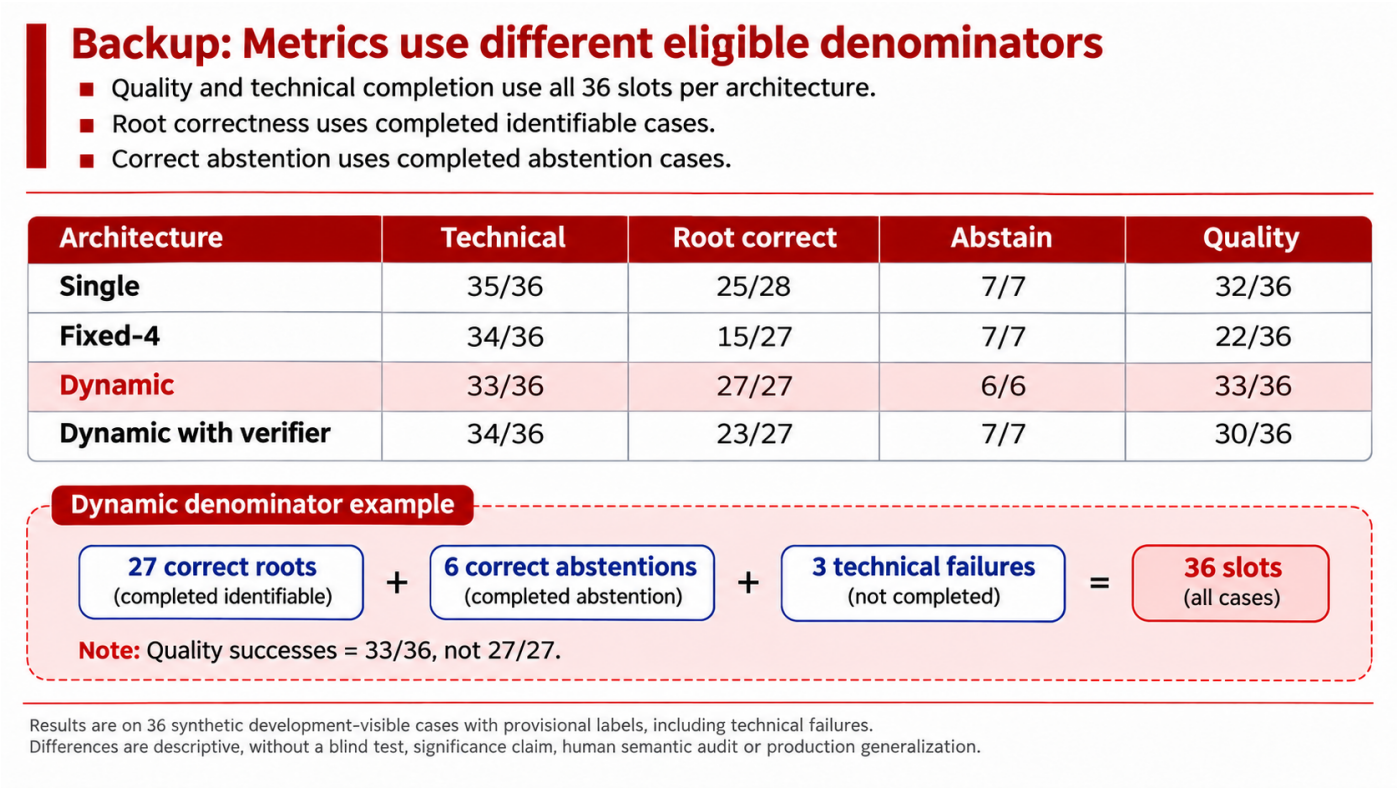
Presenter script

The coordinator chooses among investigate, finalize and abstain. Finalize asks the runtime to evaluate a selected hypothesis. It does not declare that hypothesis correct.

A resolved report contains an accepted, cited root cause. An inconclusive report can contain a provisional explanation, clearly marked as unconfirmed. Evidence insufficiency is a normal abstention. Budget exhaustion and unrecoverable execution errors are runtime failures. Every terminal path passes through a common shutdown step before report rendering, so the trace preserves why the run stopped.

Slide 17: Metrics use different eligible denominators

Backup: use during questions - Speaker block: Q&A backup



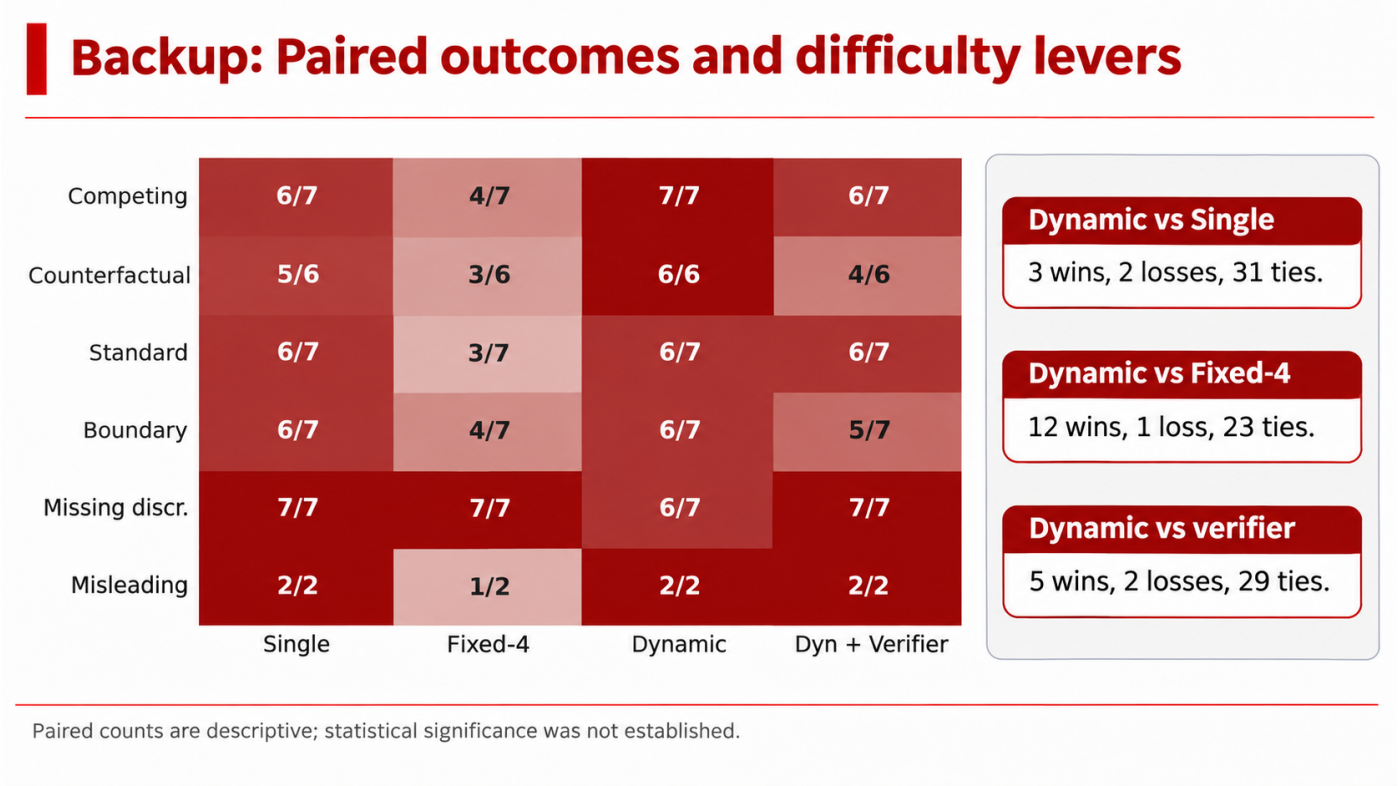
Presenter script

The denominator depends on the question. Technical completion counts completed runs out of all thirty-six slots. Quality success also uses all thirty-six, so failed executions cannot disappear from the score.

Root correctness considers only completed identifiable cases. Correct abstention considers only completed cases whose evidence should require abstention. For Dynamic, twenty-seven correct roots plus six correct abstentions give thirty-three quality successes. Three technical failures remain in the overall denominator. This is why its conditional root correctness and overall quality success are different figures.

Slide 18: Paired outcomes and difficulty levers

Backup: use during questions - Speaker block: Q&A backup



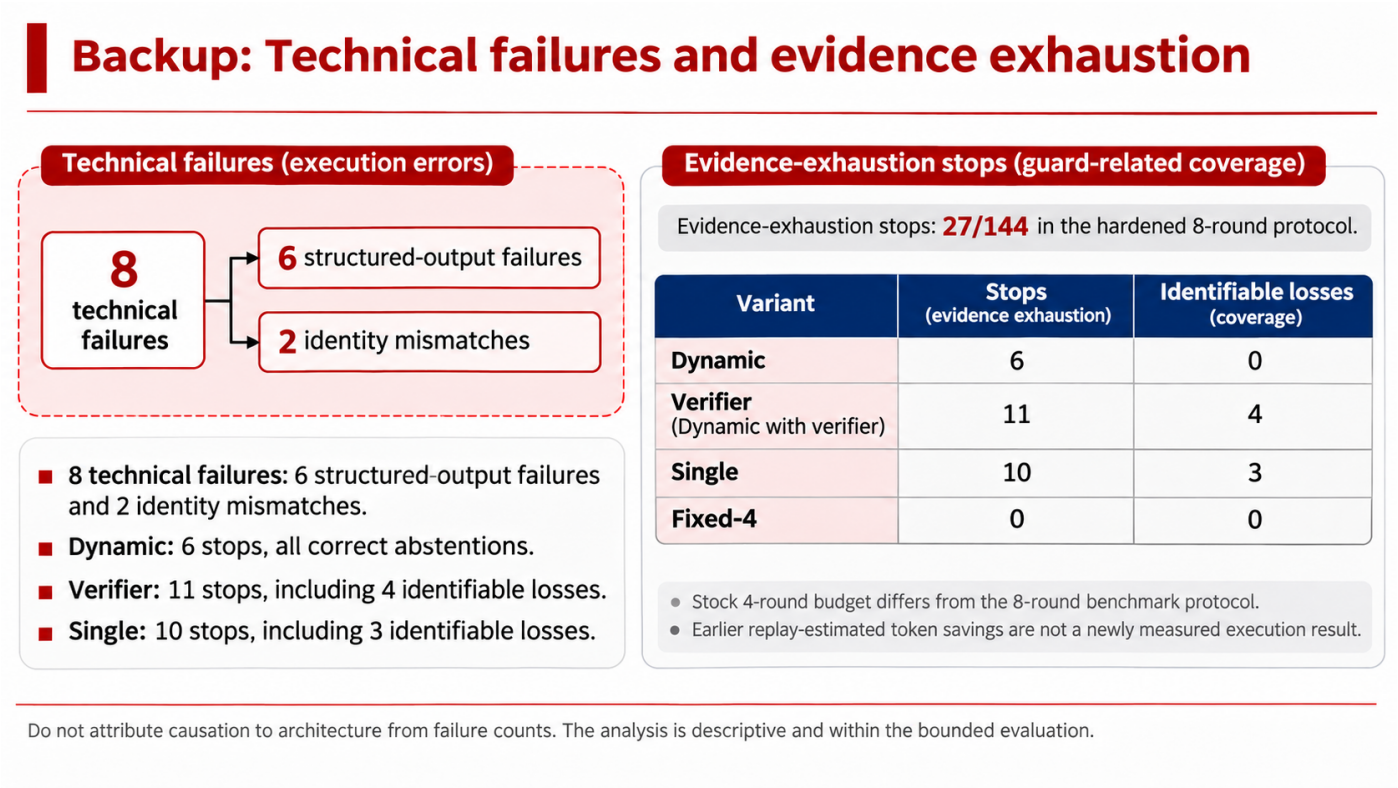
Presenter script

These counts compare the architectures on the same cases. Against Single, Dynamic has three wins, two losses and thirty-one ties. Against Fixed-Four, it has twelve wins, one loss and twenty-three ties. Against the verifier variant, it has five wins, two losses and twenty-nine ties.

The lever table shows where the differences occur. Dynamic succeeds on all seven competing-cause cases and all six counterfactual cases in this batch. Most paired outcomes are ties. These descriptive counts do not establish a statistically significant advantage.

Slide 19: Technical failures and evidence exhaustion

Backup: use during questions - Speaker block: Q&A backup



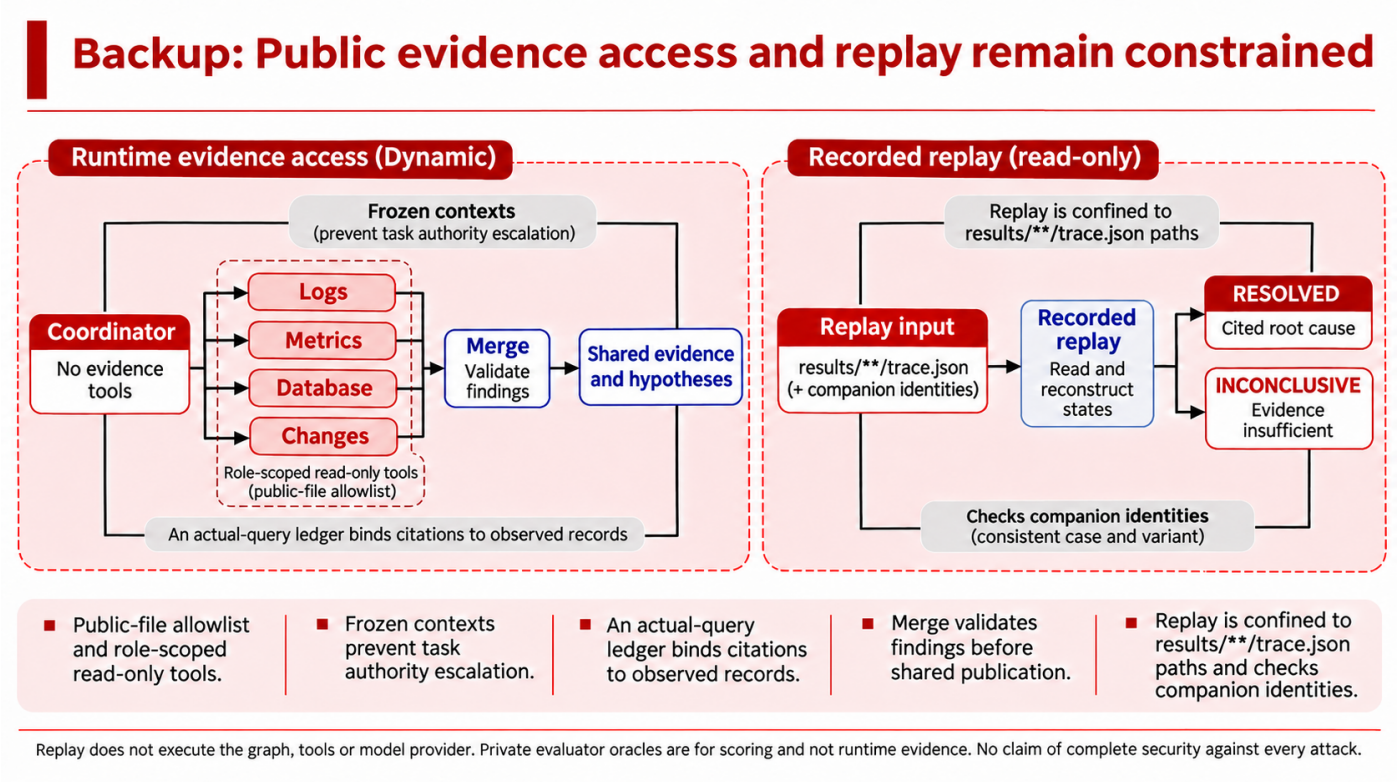
Presenter script

The eight technical failures contain six structured-output failures and two finding-identity mismatches. They remain in the quality denominator. Their presence across the variants does not establish which architecture caused them.

The evidence-exhaustion guard fired in twenty-seven traces in the hardened eight-round protocol. Dynamic's six stops were correct abstentions. However, the verifier variant lost four identifiable cases and Single lost three at this stop condition. The guard therefore needs coverage analysis as well as cost analysis. Earlier token-saving estimates from replay remain estimates, rather than newly measured execution savings.

Slide 20: Public evidence access and replay remain constrained

Backup: use during questions - Speaker block: Q&A backup



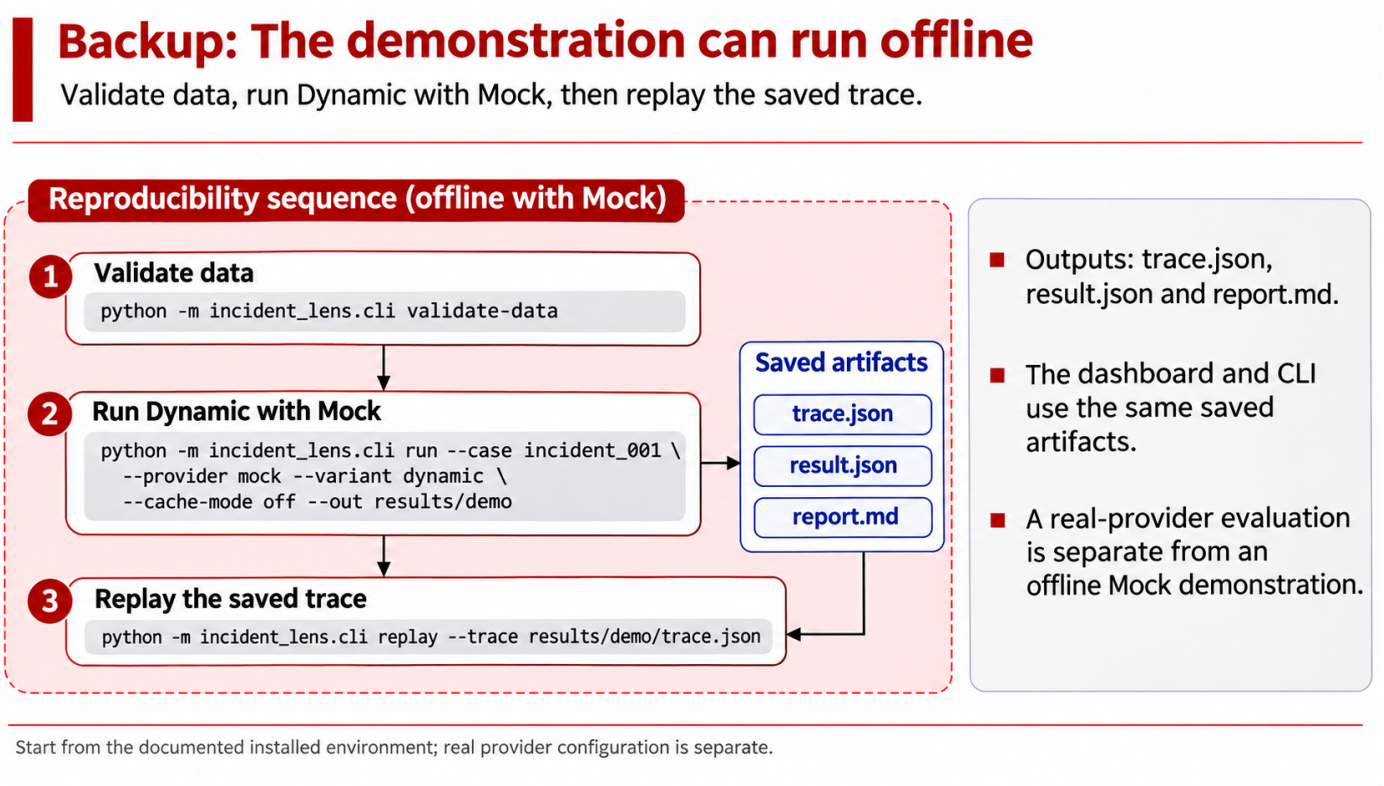
Presenter script

At runtime, evidence access is limited to approved public files and the tools allowed for the role. Frozen contexts prevent a task from raising its authority. The observation ledger records what the specialist actually queried, and Merge validates citations before sharing the findings.

Replay has a separate boundary. It opens saved trace files within the results directory and checks companion metadata identities. It does not call the provider again. These controls support provenance and confinement. They are implementation properties, rather than a claim of complete security against every possible attack.

Slide 21: The demonstration can run offline

Backup: use during questions - Speaker block: Q&A backup



Presenter script

The offline path starts with installation and data validation, then runs the Dynamic architecture with the Mock provider. The run produces a trace, result and report. That trace can then be loaded through the replay command or dashboard.

The real-provider path requires separate configuration. For a classroom demonstration, the offline run and a saved replay provide a reproducible fallback. A successful Mock demonstration establishes the implemented workflow. The real-model comparison reported earlier comes from its separate recorded evaluation batch.